

Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Overview

This homework implements and compares different reduction algorithms for summing a 40MB floating-point vector: 1. CPU baseline reduction 2. GPU atomic cascaded reduction (from HW7) 3. Harris cascaded algorithm with `__threadfence()` optimization

GPU Launch Configuration

Similar to HW7, we calculate the optimal launch configuration based on GPU properties:

$$\text{BLOCK_SIZE} = \frac{\text{max threads per SM}}{\text{max blocks per SM}}$$

$$\text{GRID_SIZE} = \text{number of SMs} \times \text{max blocks per SM}$$

This ensures maximum occupancy with a fixed number of blocks equal to the number of active blocks on the GPU.

Vector Size Calculation

$$\text{num_elements} = \frac{40 \text{ MB} \times 1024 \times 1024}{4 \text{ Bytes}} = 10,485,760 \text{ floats}$$

Task 1: CPU Reduction (Baseline)

A simple sequential sum over all elements. This serves as: - Correctness verification for all GPU implementations - Performance baseline for speedup calculations

The CPU implementation is straightforward:

```
for (int i = 0; i < elements; i++) {
    *reduction_out += vector_in[i];
}
```

Task 2: GPU Atomic Cascaded Reduction (from HW7)

This is the fastest version from the previous homework, using: - **Thread-level coarsening**: Each thread processes multiple elements via grid-stride loop - **Block-level reduction**: Using `atomicAdd` to shared memory - **Global reduction**: Single `atomicAdd` per block to global memory

The limitation is that `atomicAdd` creates serialization at the block level.

Task 3: Harris Cascaded Algorithm with threadfence

This implementation follows the Mark Harris optimization pattern with all optimizations discussed in Lecture 8:

Optimizations Applied:

1. **Grid-stride loop (Thread Coarsening)**
 - Each thread processes multiple elements: `for (i = idx; i < n; i += stride)`
 - Reduces number of threads needed and improves arithmetic intensity
2. **Warp Shuffle Reduction**
 - Uses `__shfl_down_sync()` for intra-warp reduction
 - No shared memory bank conflicts
 - No synchronization overhead within warp

```
for (int offset = WARP_SIZE / 2; offset > 0; offset /= 2) {
    val += __shfl_down_sync(0xffffffff, val, offset);
}
```
3. **Block-level Reduction**
 - First reduces within each warp using shuffle
 - Warp leaders write to shared memory
 - First warp reduces all partial sums
4. **__threadfence() for Final Reduction**
 - Ensures memory visibility across blocks
 - Allows single-kernel reduction without second kernel launch
 - Uses atomic counter to detect last block:

```
__threadfence();
unsigned int old_count = atomicInc(&block_counter, num_blocks);
if (old_count == num_blocks - 1) {
    // Last block performs final reduction
}
```
5. **Loop Unrolling (Task 4)**
 - Processes 4 elements per iteration to improve instruction-level parallelism
 - Reduces loop overhead

Why threadfence is Important

The `__threadfence()` function ensures that: - All writes to global/shared memory are visible to all other threads - Necessary before incrementing the block counter - Without it, the last block might read stale partial sums

Comparison: Atomic vs threadfence Approach

Aspect	Atomic Cascaded	Harris with threadfence
Synchronization	atomicAdd to global	<code>__threadfence()</code> + atomic counter
Memory Access	Serialized atomic ops	Single write per block
Final Reduction	Sequential atomic adds	Parallel read by last block
Kernel Launches	1	1

Performance Analysis

Expected Results

- **CPU:** Slowest due to sequential execution
- **GPU Atomic Cascaded:** Significant speedup but limited by atomic contention
- **Harris Cascaded:** Fastest due to:
 - Reduced atomic operations (only counter increment)
 - Warp shuffle eliminates shared memory overhead
 - Better memory coalescing

Speedup Factors

1. **Thread coarsening** improves arithmetic intensity
2. **Warp shuffle** eliminates shared memory bank conflicts
3. **Single threadfence** replaces multiple atomicAdd operations
4. **Fixed block count** ensures all SMs are utilized without over-subscription

Conclusion

The Harris cascaded algorithm with `__threadfence()` optimization represents the most efficient single-kernel reduction approach because: 1. It minimizes synchronization overhead 2. Uses hardware-efficient warp shuffle instructions 3. Avoids excessive atomic operations 4. Completes the entire reduction in a single kernel launch

This makes it significantly faster than both the CPU baseline and the atomic-based GPU implementation from HW7.